

Föreläsning 12

Sekvensiell logik

Introduktion

Kurs 1TE624 Elektronik I

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Använda tillstånd

Praktisk implementering

Uwe Zimmermann
Fasta tillståndets elektronik
Uppsala universitet



Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Intern variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Oanvända tillstånd

Praktisk implementering

Från förra föreläsningen ska ni komma ihåg

- problemen med rekursion och fördröjningstider i kombinatoriska nät
- att det finns olika typer av *vippor*
 - RS-vippor med direkt-verkande ingångar
 $S = 1$ sätter och $R = 1$ nollställer vippan
 - grindade vippor
 - klockade vippor
 - JK-vippor
 $J = 1$ sätter och $K = 1$ nollställer vippan
 - D-vippor
 $D = 1$ sätter och $D = 0$ nollställer vippan
 - T-vippor
vippan växlar mellan lägen med varje taktpuls
 - monostabila vippor
 - astabila vippor



Mål för dagens föreläsning.

- 1 Komplexare asynkrona nät
- 2 Synkrona sekvensiella nät
- 3 Tillståndsgrafer
- 4 Tillståndstabeller

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

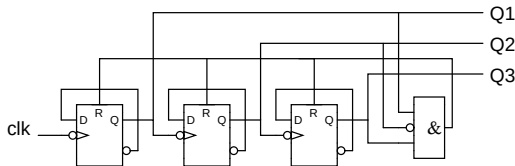
Det kompletta systemet

Använda tillstånd

Praktisk implementering



Asynkron modulo-5 räknare

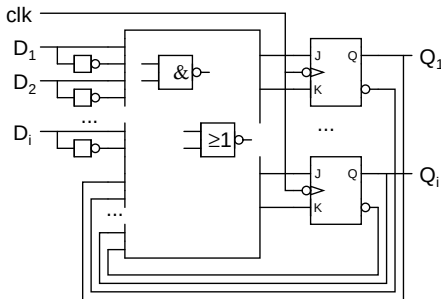


- Räknaren ska räkna från 0 till 4, dvs genom *fem* lägen $000_2 \rightarrow 001_2 \rightarrow 010_2 \rightarrow 011_2 \rightarrow 100_2 \rightarrow 000_2$
- Den kombinatoriska logiken (AND-grinden till höger) jämför räknarens utgångar med värdet $101_2 = 5_{10}$
- Vid räknarställningen $101_2 = 5_{10}$ nollställs räknaren
- Under grindens fördröjningstid kommer utgången visar det sjätte läget $101_2 = 5_{10}$
- Kretsen är användbar om fördröjningstiden är kort jämfört med pulstiden



- Asynkrona nät består av
 - ingångar
 - sekvensiella kopplingar med vippor, t.ex. asynkrona räknare
 - kombinatorisk logik
 - utgångar
- utgångssignaler och interna signaler ändras asynkront
- felaktiga signaler kan uppstå om två ingångar ändras samtidigt





- *Synkrona nät* består av
 - ingångar
 - ett kombinatoriskt nät
 - utgångar
- utgångarna är kopplade via klockade vippor med en gemensam, synkron taktsignal
- utgångarna kan återkopplas till ingångar av det kombinatoriska nätet
- varje unik signalkombination i nätet kallas för ett *tillstånd*



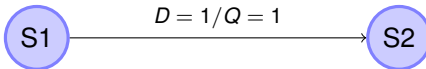
- I en *tillståndsgraf* visualiseras *tillstånden* och *övergångarna mellan olika tillstånden* i ett synkront nät.
- Varje tillstånd symboliseras av en *node*



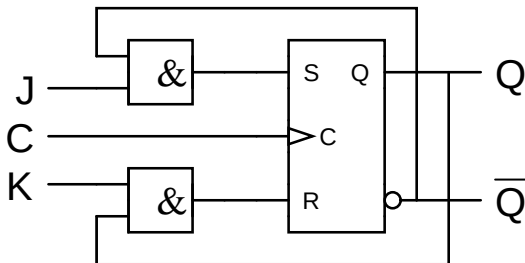
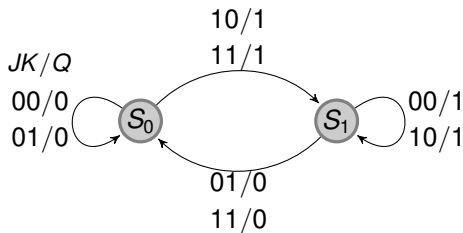
- Övergången mellan två tillstånd symboliseras av en pil



- Ingångs- och utgångssignalerna noteras vid pilen



Tillståndsgraf av en JK-vippa



Repetition

Mål

Komplexare asynkrona
nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Öarvända tillstånd

Praktisk implementering

- Tillståndsgrafer är ett bra verktyg för att planera ett synkront nät.
- Informationen i tillståndsgrafen är dock svårt att implementera direkt.
- Informationen överförs därför oftast i en tillståndstabelle



Exempel: En synkron modulo-5 räknare

- En modulo-5 räknare delar en inkommande taktfrekvens med 5.
- Räknaren har 5 tillstånd S_0 , S_1 , S_2 , S_3 och S_4 .
- I ett tillstånd är kretsens utgång aktiv $Q = 1$.
- Den här kretsen ska ha
 - en taktingång *clock*
 - en styringång **Next**
 - en återställningsingång **Reset**
- Används i dekadiska räknare.



Tillståndsgraf av en synkron modulo-5 räknare

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

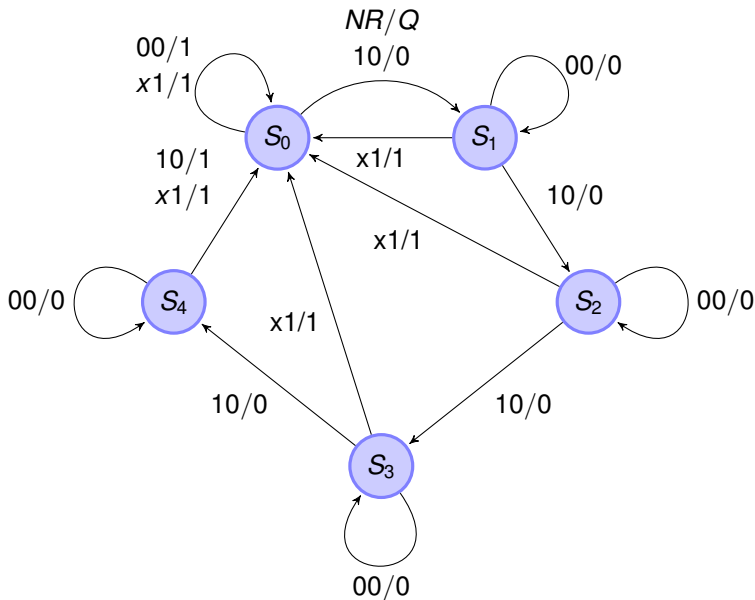
Implementering

Det kombinatoriska nätet

Det kompletta systemet

Oanvända tillstånd

Praktisk implementering



Tillståndstabell för en synkron modulo-5 räknare

nuvarande tillstånd	ingångar		följande tillstånd	utgång Q
	N(ext)	(R)eset		
S_0	0	0	S_0	1
S_0	0	1	S_0	1
S_0	1	0	S_1	0
S_0	1	1	S_0	1
S_1	0	0	S_1	0
S_1	0	1	S_0	1
S_1	1	0	S_2	0
S_1	1	1	S_0	1
S_2	0	0	S_2	0
S_2	0	1	S_0	1
S_2	1	0	S_3	0
S_2	1	1	S_0	1
S_3	0	0	S_3	0
S_3	0	1	S_0	1
S_3	1	0	S_4	0
S_3	1	1	S_0	1
S_4	0	0	S_4	0
S_4	0	1	S_0	1
S_4	1	0	S_0	1
S_4	1	1	S_0	1

Vi kan förenkla denna tabell lite...

Repetition

Mål

 Komplexare asynkrona
nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Använda tillstånd

Praktisk implementering



Förenklad tillståndstabell för en synkron modulo-5 räknare

nuvarande tillstånd	ingångar		följande tillstånd	utgång Q
N(ext)	(R)eset			
S_0	0	0	S_0	1
	1	0	S_1	0
	x	1	S_0	1
S_1	0	0	S_1	0
	1	0	S_2	0
	x	1	S_0	1
S_2	0	0	S_2	0
	1	0	S_3	0
	x	1	S_0	1
S_3	0	0	S_3	0
	1	0	S_4	0
	x	1	S_0	1
S_4	0	0	S_4	0
	1	0	S_0	1
	x	1	S_0	1



- Allmänt gäller att n interna variabler behövs för att hantera mellan $2^{n-1} + 1$ och 2^n enskilda tillstånd
- För att kunna hantera 5 olika tillstånd $S_0 \dots S_4$ behöver vi därmed 3 interna variabler: A, B, C
- Kopplingen mellan tillstånden och de interna variablerna är i princip godtycklig
 - nätet blir mindre komplext om man kan använda de interna variablerna som utgångsvariabler
 - nätet kan även blir mindre komplext för vissa kombinationer mellan tillstånden och variablerna
- Vid färre än 2^n tillstånd i nätet måste man ta hänsyn till de oanvända kombinationerna av variablerna
- Interna variabler implementeras med hjälp av bistabila vippor



Introduktion av interna variabler

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Använda tillstånd

Praktisk implementering



tillstånd	variabler		
	A	B	C
S ₀	0	0	0
S ₁	0	0	1
S ₂	0	1	0
S ₃	0	1	1
S ₄	1	0	0

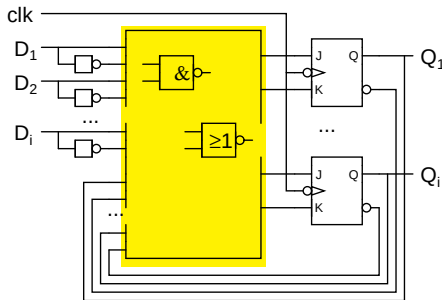
nuvarande			ingångar		följande			utgång
A	B	C	N(ext)	(R)eset	A	B	C	Q
0	0	0	0	0	0	0	0	1
			1	0	0	0	1	0
			x	1	0	0	0	1
0	0	1	0	0	0	0	1	0
			1	0	0	1	0	0
			x	1	0	0	0	1
0	1	0	0	0	0	1	0	0
			1	0	0	1	1	0
			x	1	0	0	0	1
0	1	1	0	0	0	1	1	0
			1	0	1	0	0	0
			x	1	0	0	0	1
1	0	0	0	0	1	0	0	0
			1	0	0	0	0	1
			x	1	0	0	0	1
1	0	1	x	x	x	x	x	x
1	1	0	x	x	x	x	x	x
1	1	1	x	x	x	x	x	x

nuvarande tillstånd Q_n	kontrollgångar		följande tillstånd Q_{n+1}
	J	K	
0	0	X	0
0	1	X	1
1	X	1	0
1	X	0	1

- Vi betraktar igen JK-vippans egenskaper
- Vilka signaler behövs för att vippan ska gå från ett läge till ett annat läge?
 - från $Q_n = 0$ till $Q_{n+1} = 0$ måste $J = 0$
 - från $Q_n = 0$ till $Q_{n+1} = 1$ måste $J = 1$
 - från $Q_n = 1$ till $Q_{n+1} = 0$ måste $K = 1$
 - från $Q_n = 1$ till $Q_{n+1} = 1$ måste $K = 0$



Implementering av ett synkront nät



Vi söker nu efter lösningar för det kombinatoriska nätet. . .



Implementering med JK-vippor

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer
Tillståndstabeller
Interna variabler

Implementering

Det kombinatoriska nätet
Det kompletta systemet
Oanvända tillstånd
Praktisk implementering

nuvarande tillstånd			ingångar		följande tillstånd			vippornas ingångar						utgång
A	B	C	N(ext)	(R)eset	A	B	C	J _A	K _A	J _B	K _B	J _C	K _C	Q
0	0	0	0	0	0	0	0	0	X	0	X	0	X	1
			1	0	0	0	1	0	X	0	X	1	X	0
			x	1	0	0	0	0	X	0	X	0	X	1
0	0	1	0	0	0	0	1	0	X	0	X	X	0	0
			1	0	0	1	0	0	X	1	X	X	1	0
			x	1	0	0	0	0	X	0	X	X	1	1
0	1	0	0	0	0	1	0	0	X	X	0	0	X	0
			1	0	0	1	1	0	X	X	0	1	X	0
			x	1	0	0	0	0	X	X	1	0	X	1
0	1	1	0	0	0	1	1	0	X	X	0	X	0	0
			1	0	1	0	0	1	X	X	1	X	1	0
			x	1	0	0	0	0	X	X	1	X	1	1
1	0	0	0	0	1	0	0	X	0	0	X	0	X	0
			1	0	0	0	0	X	1	0	X	0	X	1
			x	1	0	0	0	X	1	0	X	0	X	1
1	0	1	x	x	x	x	x	x	x	x	x	x	x	x
1	1	0	x	x	x	x	x	x	x	x	x	x	x	x
1	1	1	x	x	x	x	x	x	x	x	x	x	x	x



Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

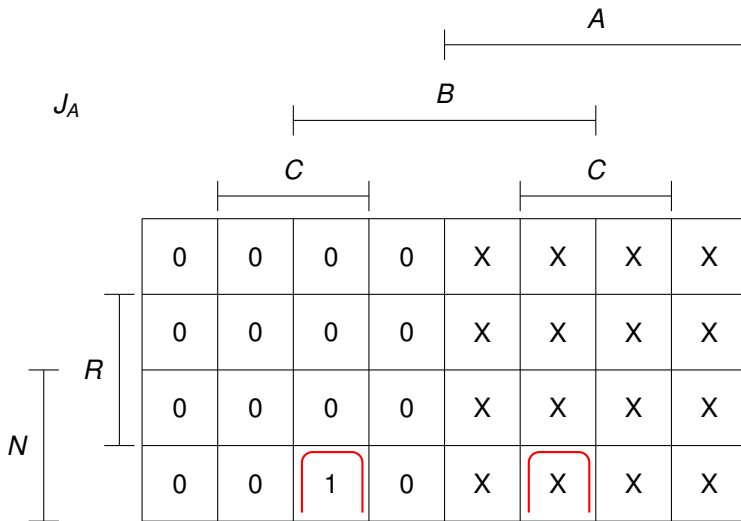
Oanvända tillstånd

Praktisk implementering

- Från tillståndstabellen får vi all information vi behöver för att ställa upp Karnaugh-diagrammen för vårt kombinatoriska nät.
- Ingångsvariablerna är ingångarna N och R , samt de interna variablerna A , B och C .
- Utgångsvariablerna är ingångarna J_i och K_i till JK-vipporna



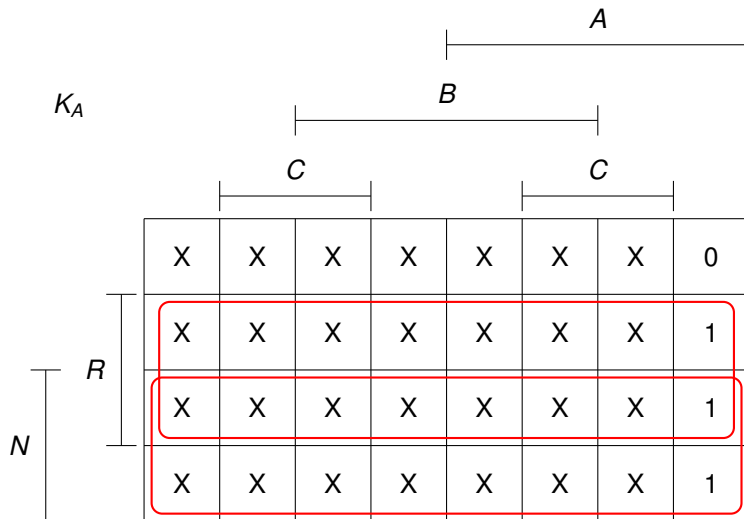
Karnaughdiagram för J_A



$$J_A = BCN\bar{R}$$



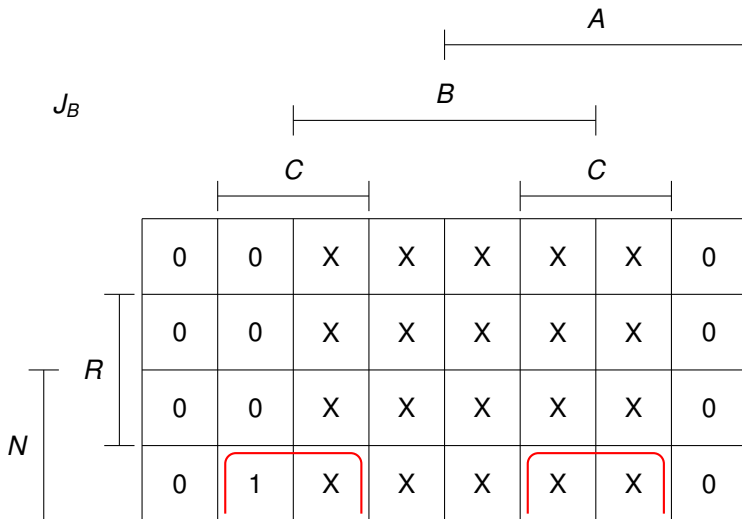
Karnaughdiagram för K_A



$$K_A = N + R$$



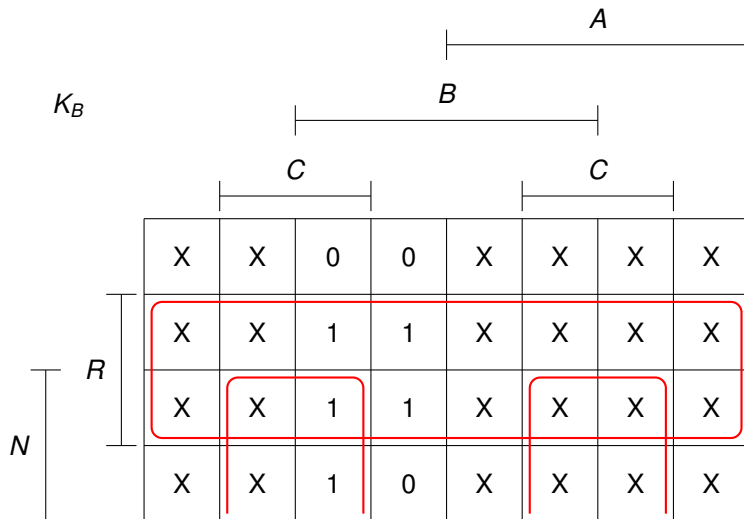
Karnaughdiagram för J_B



$$J_B = CN\bar{R}$$



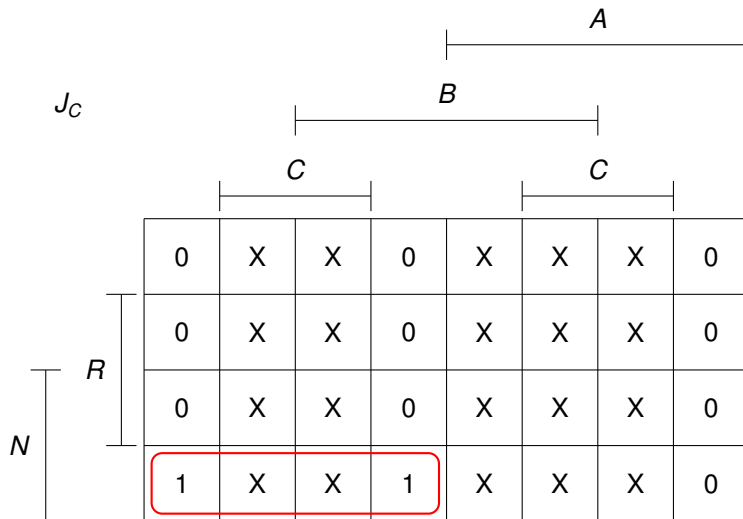
Karnaughdiagram för K_B



$$K_B = CN + R$$



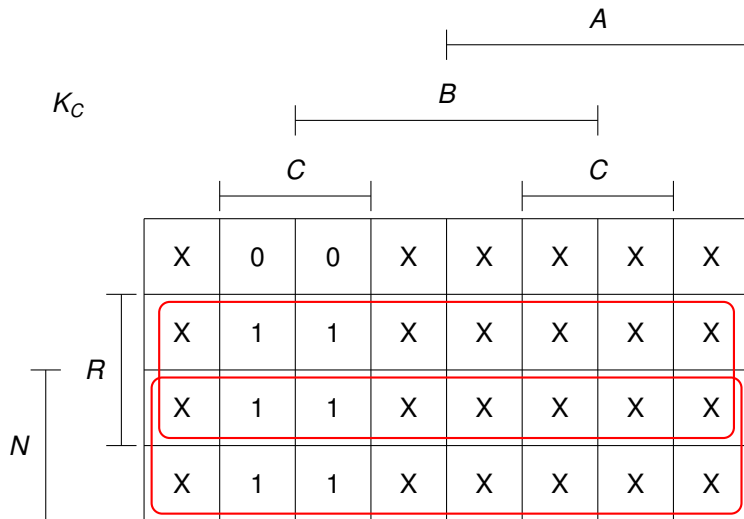
Karnaughdiagram för J_C



$$J_C = \overline{A}N\overline{R}$$



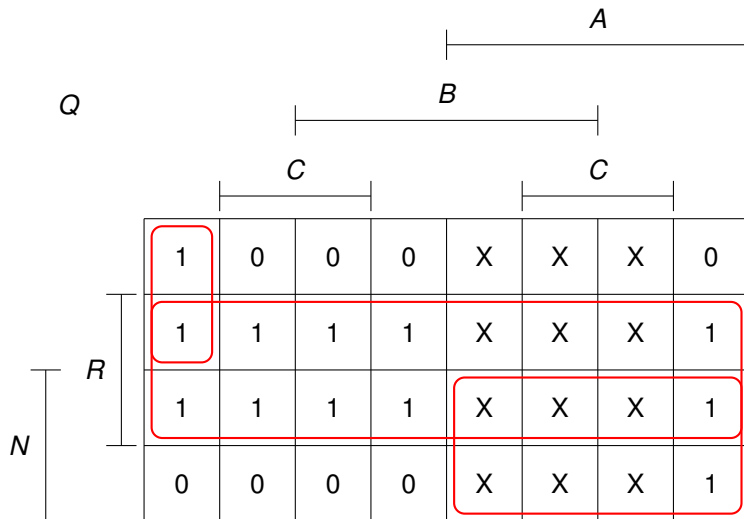
Karnaughdiagram för K_C



$$K_C = N + R = K_A$$



Karnaughdiagram för Q



$$Q = R + AN + \overline{A}\overline{B}\overline{C}\overline{N}$$



Nu har vi alla kombinatoriska ekvationer till systemet:

$$J_A = BCN\bar{R} \quad (1)$$

$$K_A = N + R \quad (2)$$

$$J_B = CN\bar{R} \quad (3)$$

$$K_B = CN + R \quad (4)$$

$$J_C = \bar{A}\bar{N}\bar{R} \quad (5)$$

$$K_C = N + R = K_A \quad (6)$$

$$Q = R + AN + \bar{A}\bar{B}\bar{C}\bar{N} \quad (7)$$



Uppbyggnaden av systemet

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

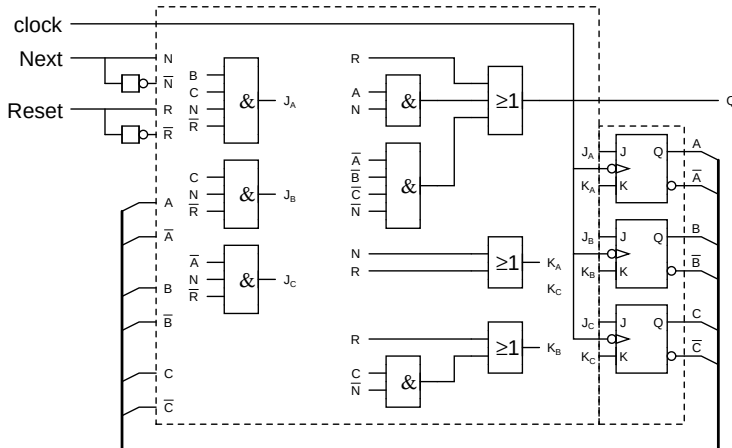
Implementering

Det kombinatoriska nätet

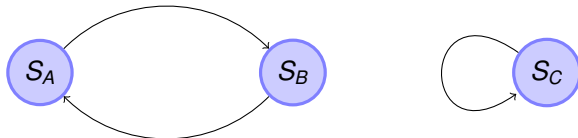
Det kompletta systemet

Använda tillstånd

Praktisk implementering



- I vår modulo-5 räknare är 3 tillstånd oanvända.
- När matningsspänningen slås på är vippornas läge odefinierad, om det inte finns en automatisk återställning *power-on reset*.
- Därför måste man säkerställa att kretsen inte är låst i en eller flera oanvända tillstånd, som t.ex.:



- Man studerar oanvända tillstånd genom att analysera de slutgiltiga kombinatoriska ekvationerna – utgående från de odefinierade lägen S_5 , S_6 och S_7 .
- Vårt modulo-5 räknare är inte låst i en oanvänd tillstånd fast hamnar alltid efter några pulser i ett definerat läge.

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Intern variabler

Implementering

Det kombinatoriska nätet

Det kompletta systemet

Oanvända tillstånd

Praktisk implementering



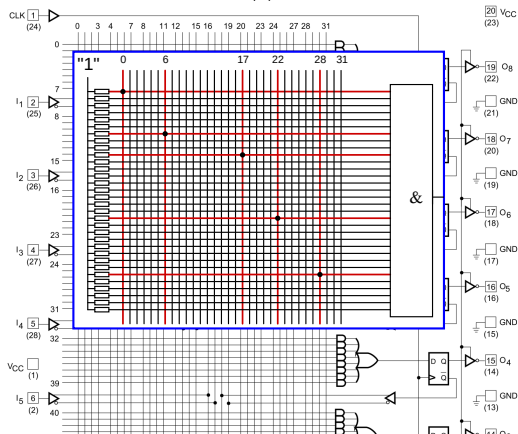
- Sekvensiella nät kan byggas med diskreta standard-grindar och vippor.
- I tidiga datorer, som t.ex. JET-80, VC-64 och de första IBM PC'or fram till 80386 användes många diskreta logikkretsar.
- Nackdelen med denna metod är längre fördröjningstider och mindre maximala taktfrekvenser.
- Diskreta logikkretsar behöver också förhållandevis stor plats på kretskorten.
- Programmerbara logikkretsar innehåller grindmatriser och registrar och kan ta upp mycket komplexa nät i en liten kapsel.
- Fördröjningstider inuti chipset är mycket korta och tillåter höga taktfrekvenser.
- Grindmatriserna omfattar mallan några hundra och flera hundratusen grindar.





LOGIC DIAGRAM

16R8 (-4)



- varje utgång O_i kan defineras som summa av upp till 8 produkttermer
- varje produktterm kan innehålla 16 individuella variabler och deras komplement

Repetition

Mål

Komplexare asynkrona nät

Synkrona nät

Tillståndsgrafer

Tillståndstabeller

Interna variabler

Implementing

Det kombinatoriska nätet

Det kompletta systemet

Oanvända tillstånd

Praktisk implementering

